Distracted Driver Detection from Dashboard Images

UE24CS352A Machine Learning - Mini-Project (Problem #24) | Arya Devang (PES1UG24AM054) | Ananthakrishna (PES1UG24AM036)

Code: https://github.com/Arya-677/ML_miniProject_sem5 (private; shared with faculty/TAs)

1. Problem statement

Given one in-cabin camera image of a driver, predict what the driver is doing: safe driving or one of nine distractions (texting or talking on the phone with the left or right hand, operating the radio, drinking, reaching behind, hair/makeup, talking to a passenger). This is a 10-class image classification problem. We re-implement the CS229 report *Machine Learning Techniques for Distracted Driver Detection* (Feng & Yue, 2019): five classifiers on raw pixels. We also ask whether their accuracy carries over to drivers never seen in training.

2. Dataset

State Farm Distracted Driver Detection (Kaggle): 22,424 labelled 640x480 RGB images in 10 classes (c0 safe driving, c1-c9 distractions), about 2,000-2,500 per class, from 26 drivers; `driver_imgs_list.csv` gives the driver of every image. The images are frames of short videos, so frames of one driver are almost identical. The official test images are unlabelled, so we hold out 20% of the labelled images for validation (this run: 17,747 training / 4,677 validation images).

3. Approach

- **Pre-processing:** resize to 64x64x3 (OpenCV, area interpolation), flatten to a 12,288-vector, scale to [0,1], subtract the training mean image (computed on training data only).
- **Models:** linear SVM (multi-class hinge loss + L2), softmax regression (cross-entropy + L2) and a two-layer network (100 ReLU units, softmax output, He initialisation, learning-rate decay) - all from scratch in NumPy; Gaussian Naive Bayes and a Decision Tree from scikit-learn with default settings.
- **Two protocols:** (i) random 80/20 split, as in the reference report; (ii) driver-wise 80/20 split in which no driver appears in both sets, which measures generalisation to new people.
- **Tuning:** learning rate and regularisation of the SVM, softmax and network are chosen by grid search on an inner split of the training data, so the validation set is used only for the final numbers.

4. Implementation overview

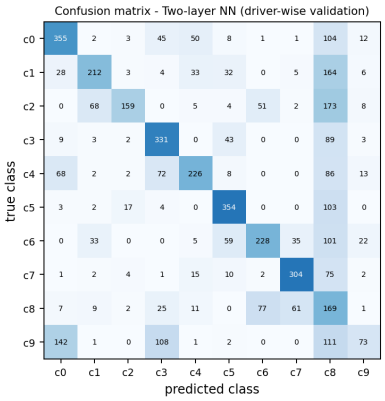
Everything is in one Python notebook (`distracted_driver_detection.ipynb`): data loading and caching, vectorised losses and back-propagation, mini-batch SGD with a fixed random seed (SVM/softmax are sensitive to initialisation), a numerical gradient check of all analytic gradients, hyper-parameter search, evaluation (accuracy, per-class accuracy, confusion matrix, loss curves), saved models and a live-demo cell that classifies any image. Results of each run are written to `results_*/results.json`.

5. Results and conclusions

Accuracy (%) on the training and validation sets; best validation result of each protocol in bold/green.

Model	Random: train	Random: val	Driver-wise: train	Driver-wise: val
Naive Bayes	55.47	55.22	61.12	32.61
Decision Tree	100.00	89.12	100.00	29.66
Linear SVM	96.16	95.34	93.63	45.39
Softmax	71.69	71.39	75.97	41.93
Two-layer NN	98.57	**98.02**	99.10	**51.55**

- On the random split, validation accuracy ranks: Two-layer NN 98.0%, Linear SVM 95.3%, Decision Tree 89.1%, Softmax 71.4%, Naive Bayes 55.2%.
- On the driver-wise split, validation accuracy ranks: Two-layer NN 51.6%, Linear SVM 45.4%, Softmax 41.9%, Naive Bayes 32.6%, Decision Tree 29.7%.
- Holding out whole drivers changes the best model (Two-layer NN) from 98.0% to 51.6%; averaged over all models, validation accuracy falls by 41.6 points.
- A large drop is what we would expect if the random split is optimistic: frames of the same driver are near-duplicates, so a model can partly memorise people instead of learning the activity.
- The best model (Two-layer NN) fits the training set at 99.1% against 51.6% on validation, a gap of 47.6 points - clear overfitting, so stronger regularisation, augmentation or a CNN would help.
- Hardest classes for Two-layer NN: Talking to passenger (16.7%) and Talking on phone - right (33.8%); easiest: Operating the radio (73.3%).
- Most frequent confusion: 'Talking on phone - right' predicted as 'Hair and makeup' (173 validation images).
- Our random-split numbers can be compared with the reference report (two-layer net 92.2%, softmax 82.3%, SVM 71.4%, decision tree 84.7%, Naive Bayes 55.0%): ours are 98.0%, 71.4%, 95.3%, 89.1% and 55.2%.



Limitations and next steps. 64x64 raw pixels lose detail (phone versus hand) and fully-connected models ignore spatial structure; results use a single seed and a single split. Next: a CNN or transfer learning, data augmentation, driver-grouped k-fold cross-validation, and semi-supervised use of the unlabelled test images.